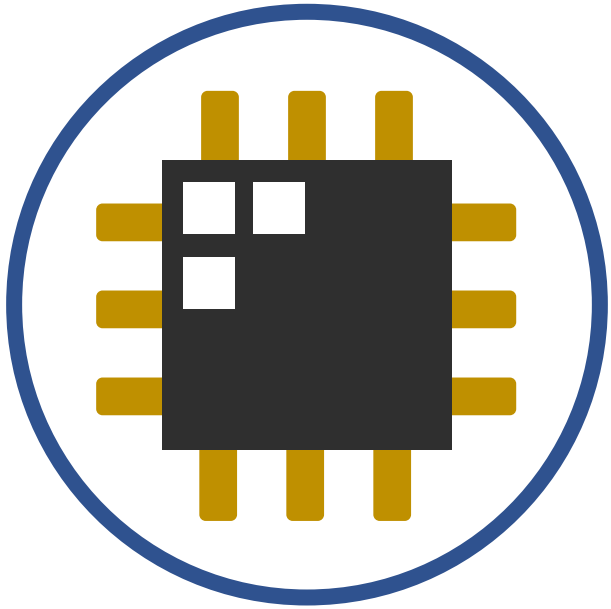
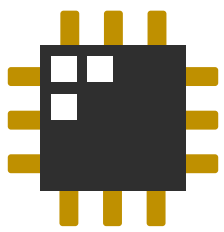




EEE 113:

Introduction to Electrical and Electronics Engineering Systems

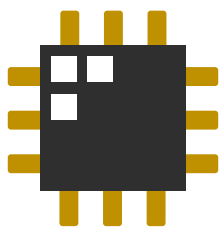
Week 3 – Memories: Flipflops/Registers



Implementing more complex functions

- Given what you know so far, implement this in hardware:

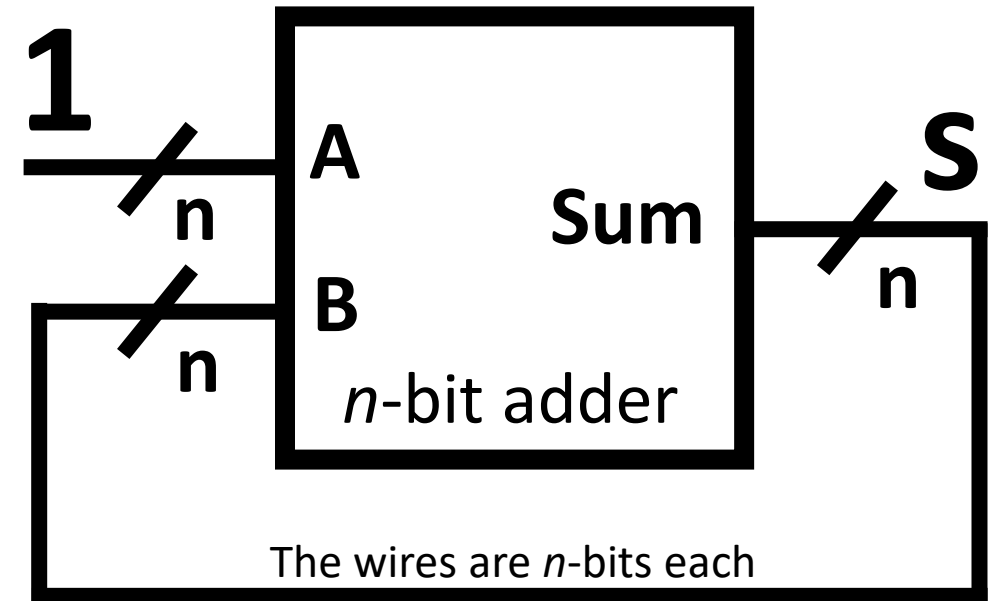
```
s = 0  
for i in range(9):  
    s = s + 1
```



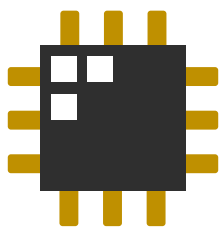
Implementing more complex functions

- Given what you know so far, implement this in hardware:

```
s = 0
for i in range(9):
    s = s + 1
```



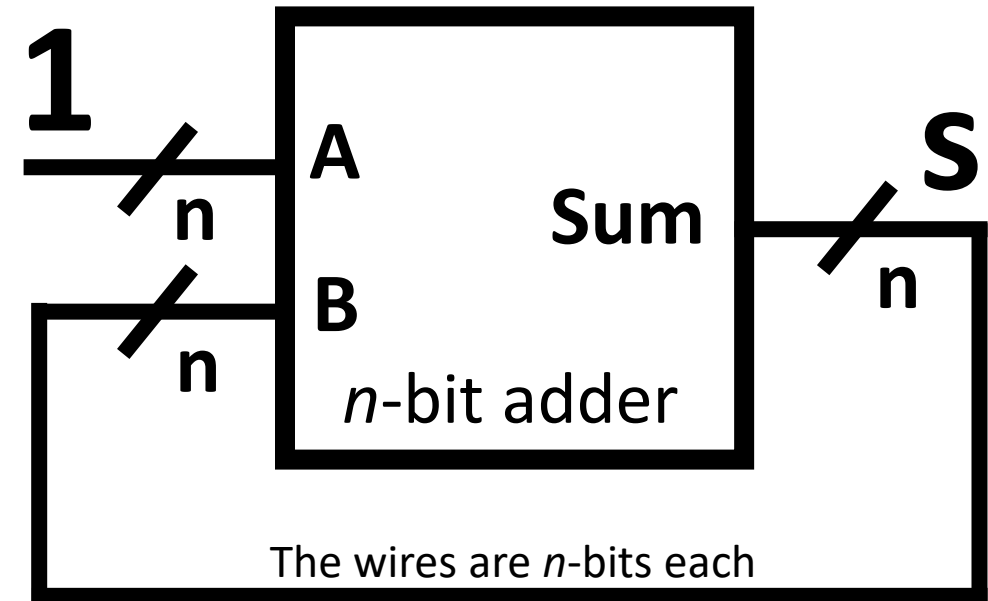
?



Implementing more complex functions

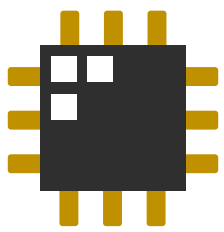
- Given what you know so far, implement this in hardware:

```
s = 0
for i in range(9):
    s = s + 1
```



Issue 1: How do you set $s = 0$ at the beginning?

Issue 2: How do you control this? You only need to do it 9 times!



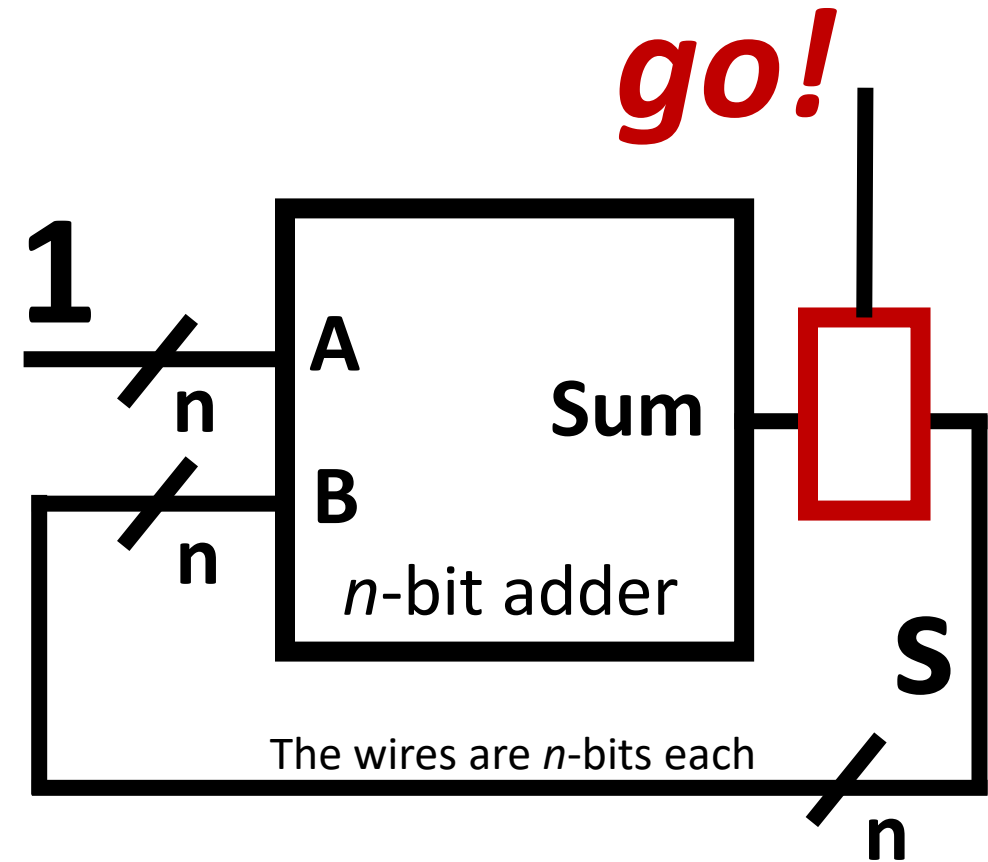
Implementing more complex functions

- We need a logic element that can:
 - **store** the current value of the output
 - be **triggered** for controlled updates
 - be **initialized** to some defined value

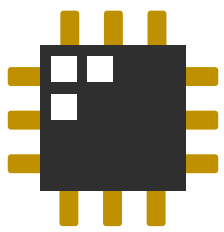
```
s = 0
```

```
for i in range(9):
```

```
    s = s + 1
```

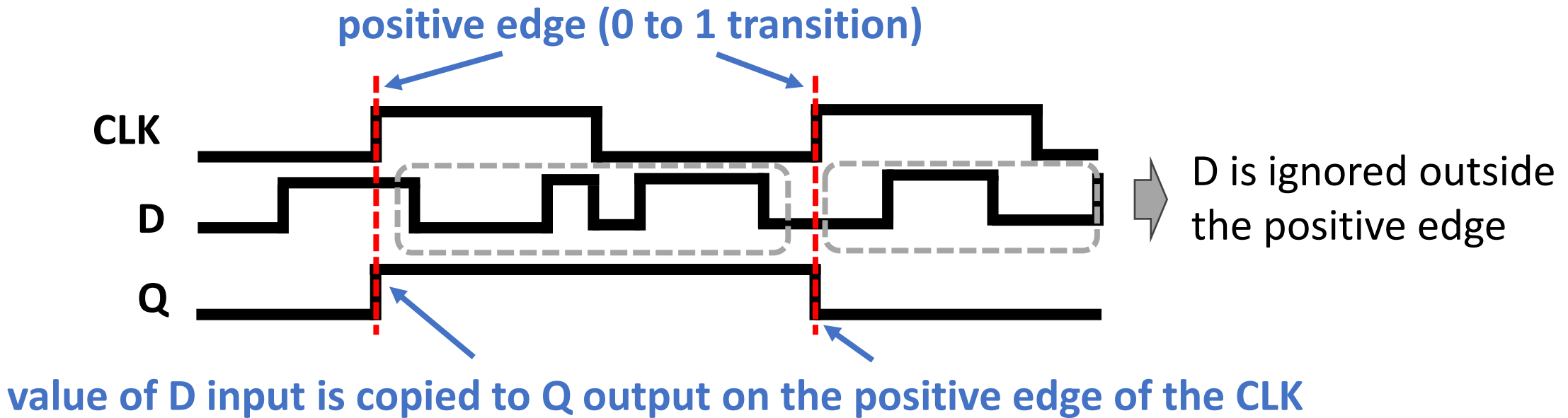
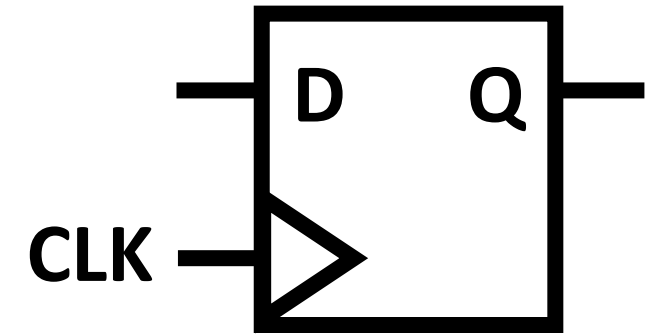


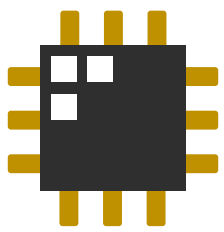
trigger the *go!* signal
9 times and you're done



Flipflops

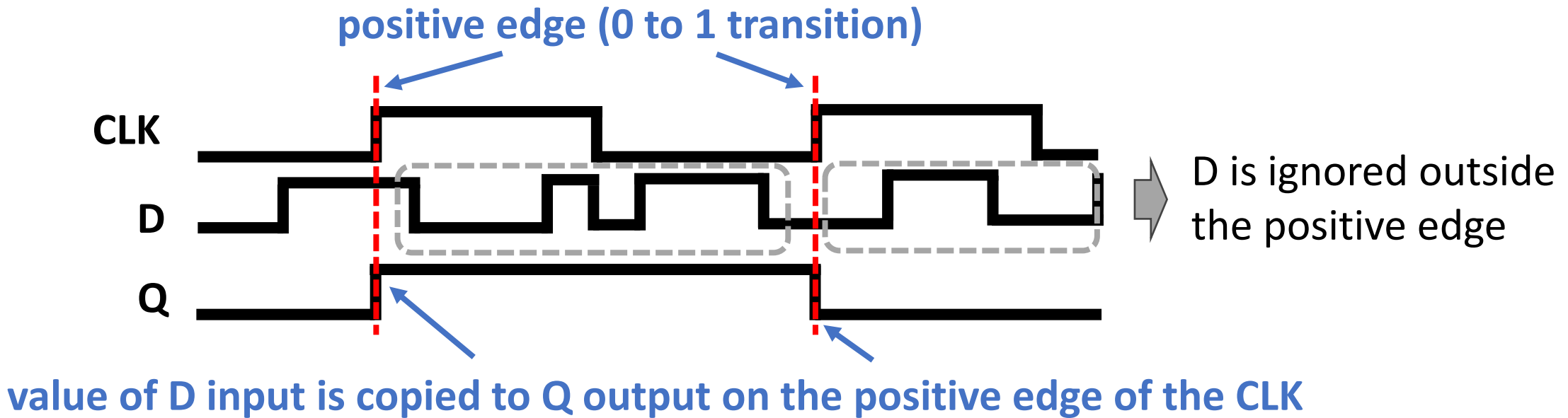
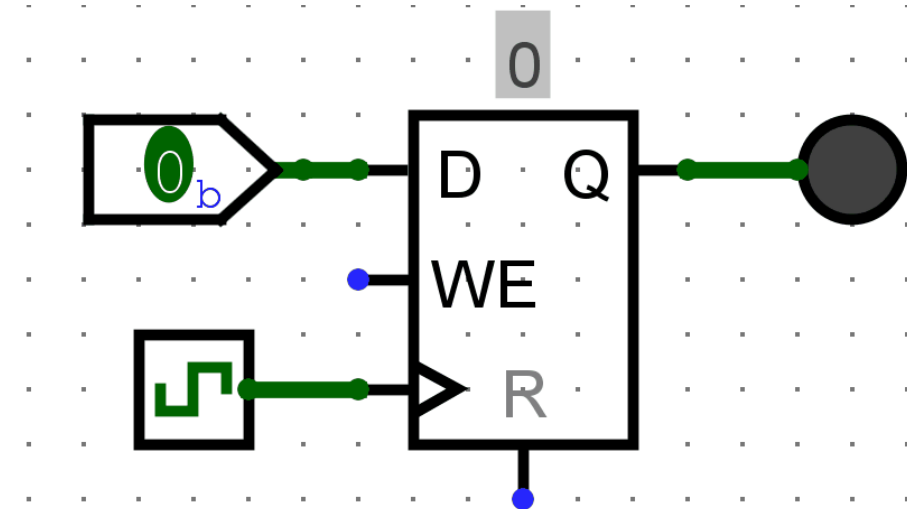
- A logic element that ***stores*** state
 - Holds a 0 or 1 at the output **Q** independent of the **D** input and is ***triggered*** to update at every rising/positive edge of the clock (**CLK**) input.
 - Has a reset input (not shown right now) allowing the output Q to be ***initialized*** to 0.

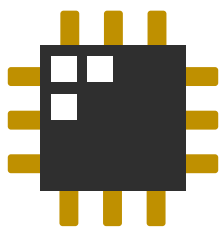




Flipflops

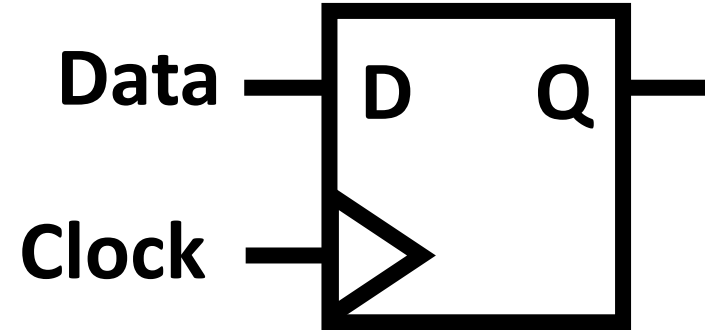
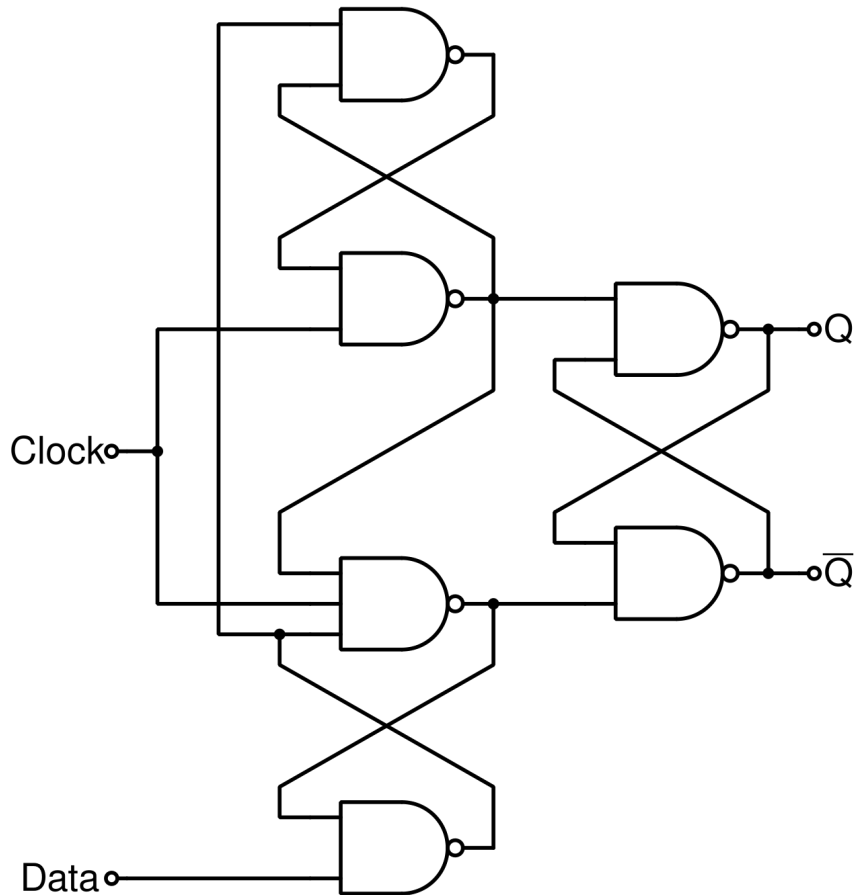
- A logic element that **stores** state
 - Holds a 0 or 1 at the output **Q** independent of the **D** input and is **triggered** to update at every rising/positive edge of the clock (**CLK**) input.
 - Has a reset input (not shown right now) allowing the output Q to be **initialized** to 0.





Flipflops

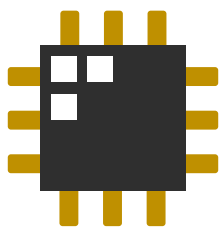
- Aside: Flipflops can be implemented using logic gates
 - Incorporating some sort of feedback enables them to **hold** values



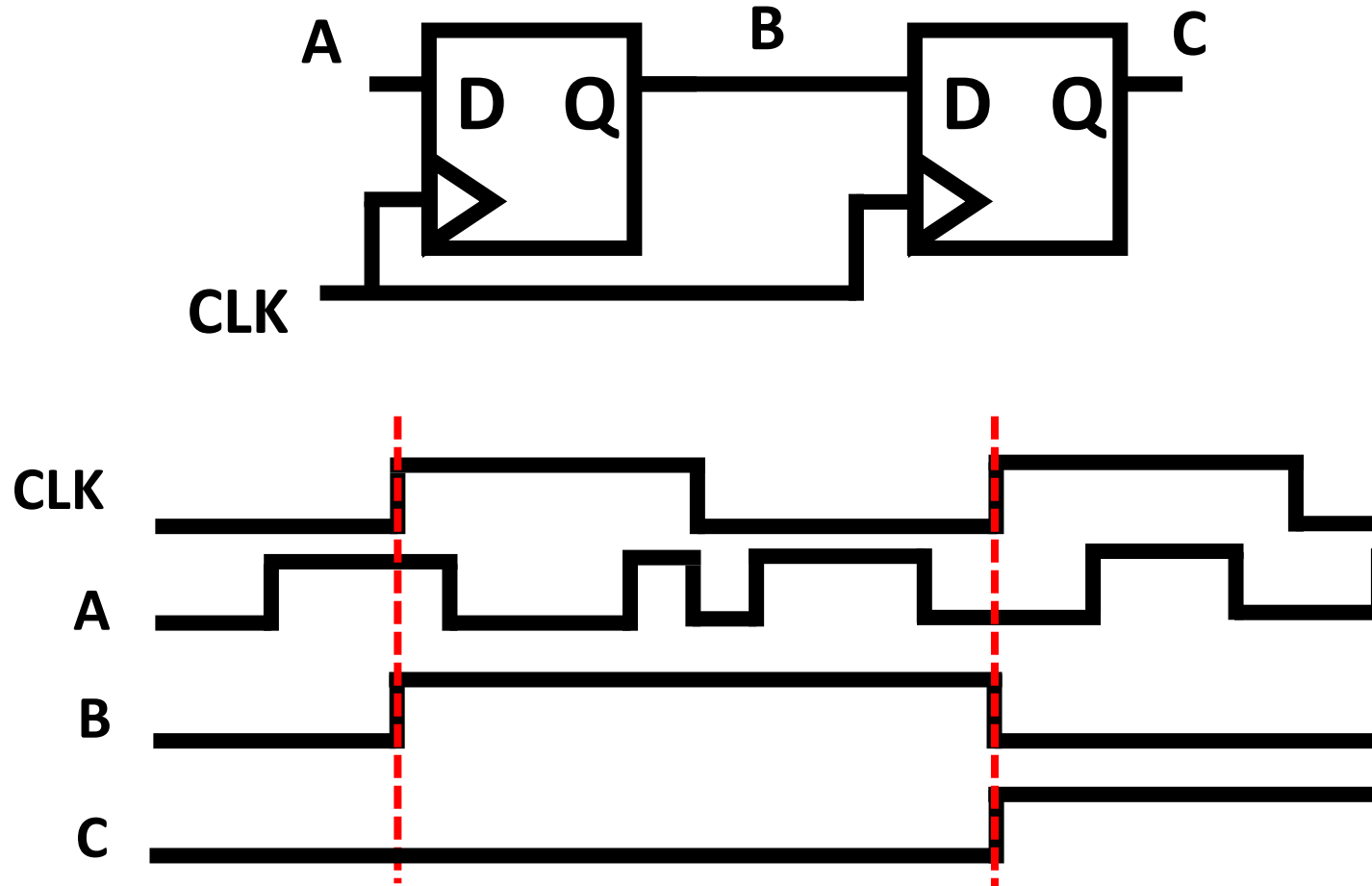
Analyzing what happens here for every change in **D (Data)** and **Clock** is a little tedious...

We can just abstract the functionality and approach it on a higher level.

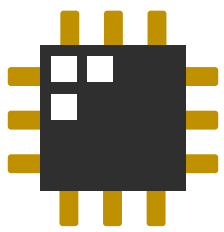
More in-depth flipflop functionality in EEE 143!



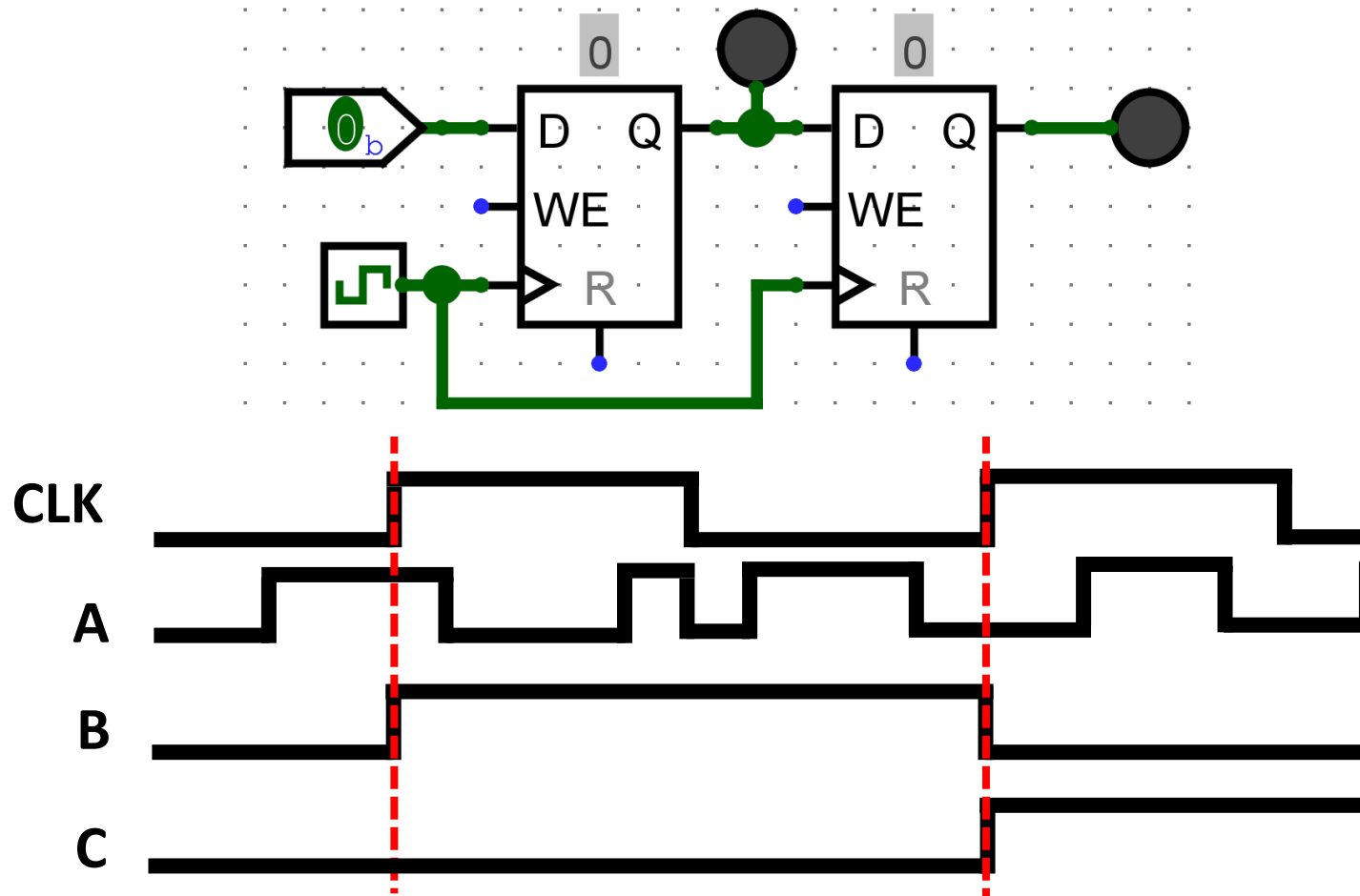
Flipflop behavior



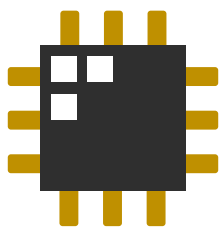
The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger.
The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.



Flipflop behavior

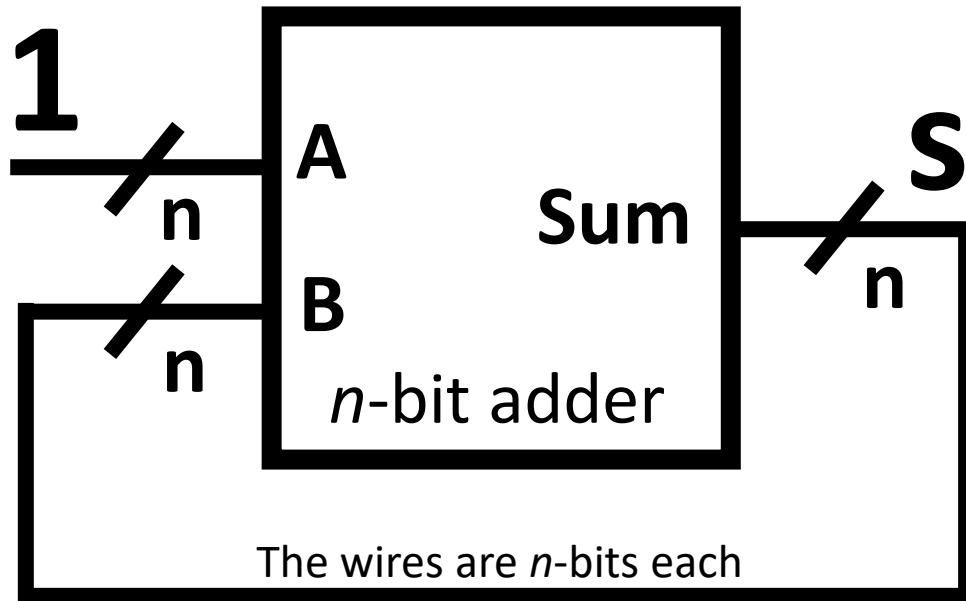


The input of the flipflop is checked *right before* ($t = 0 \text{ minus}$) the positive edge trigger. The output of the flipflop is updated *right after* ($t = 0 \text{ plus}$) the positive edge trigger.

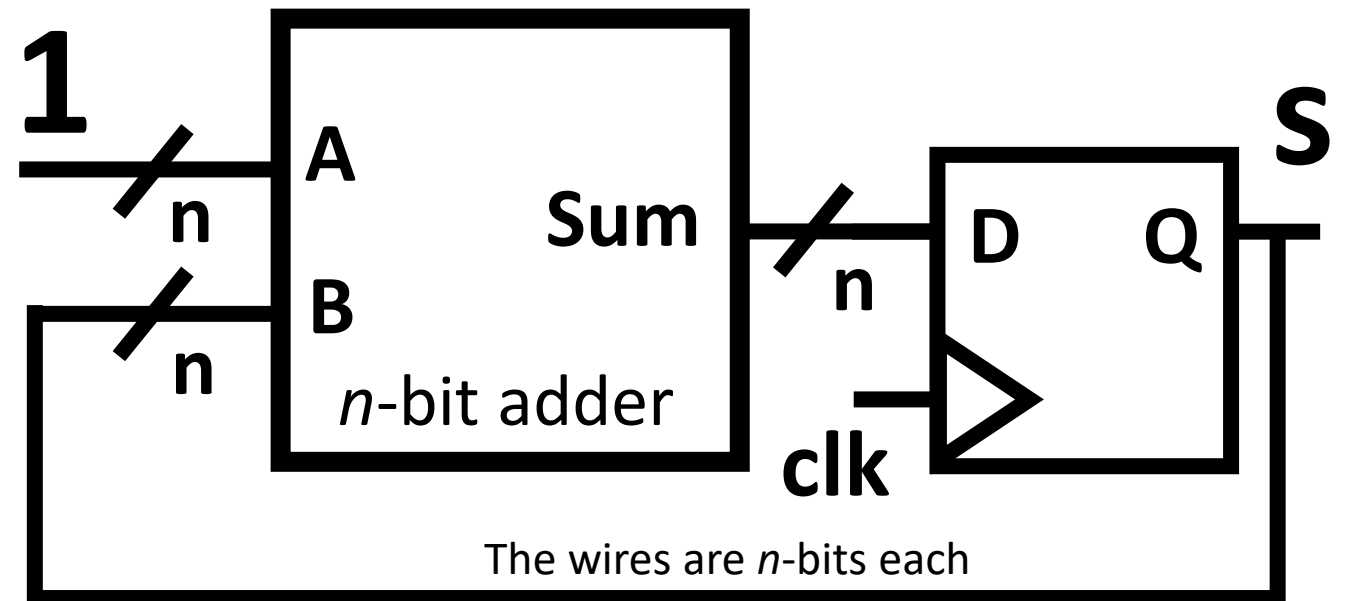


Accumulator design: Round 2

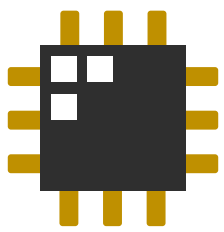
- Redesigning the accumulator



Can't be controlled!

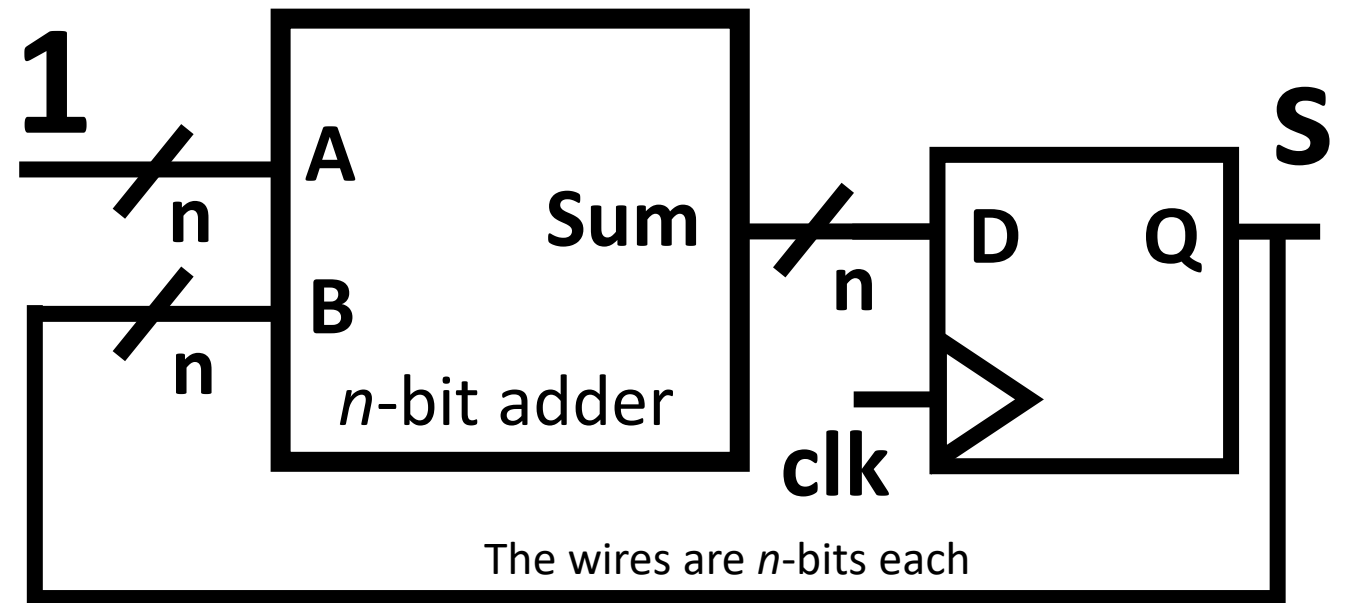
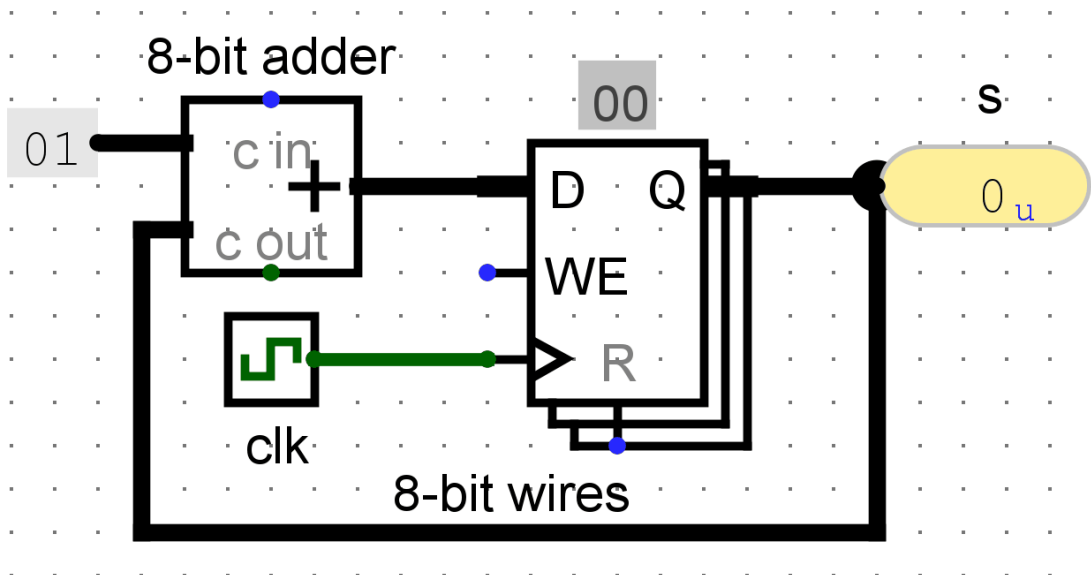


Positive edge trigger for control!

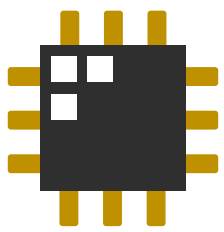


Accumulator design: Round 2

- Redesigning the accumulator

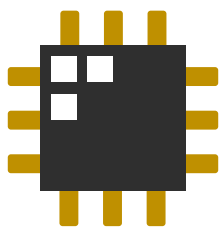


Positive edge trigger for control!



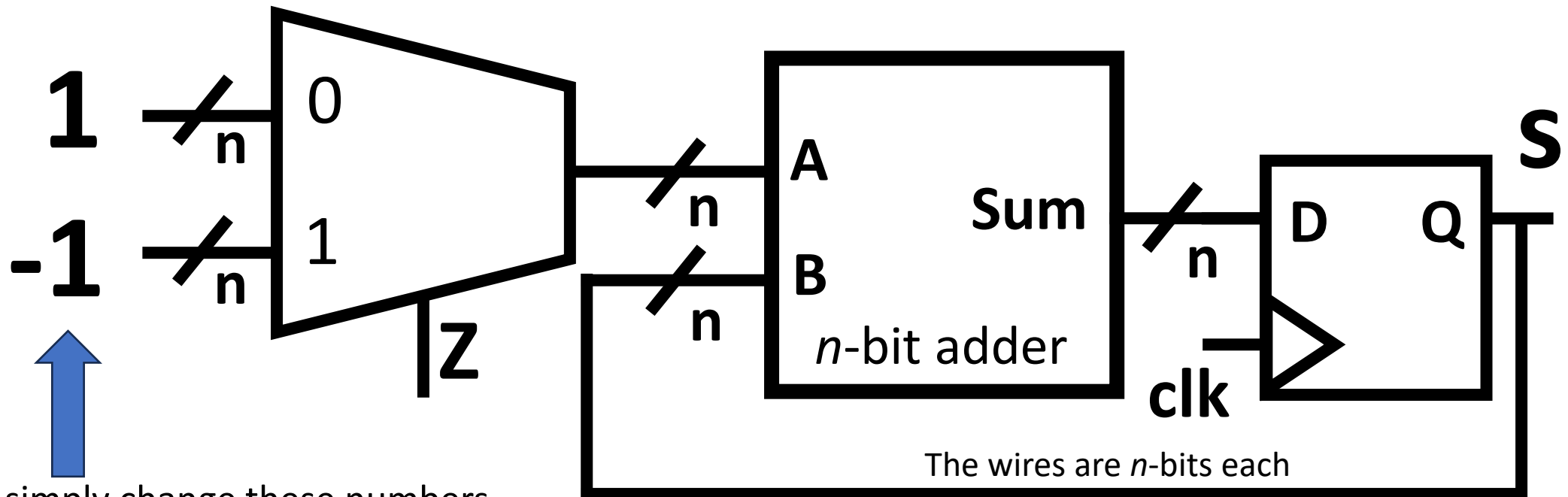
Introducing more complexity

- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)
 - We need to add some logic. *Hint*: we covered this last meeting!

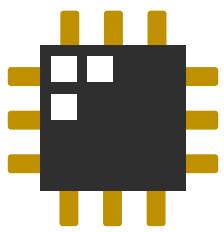


Introducing more complexity

- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)

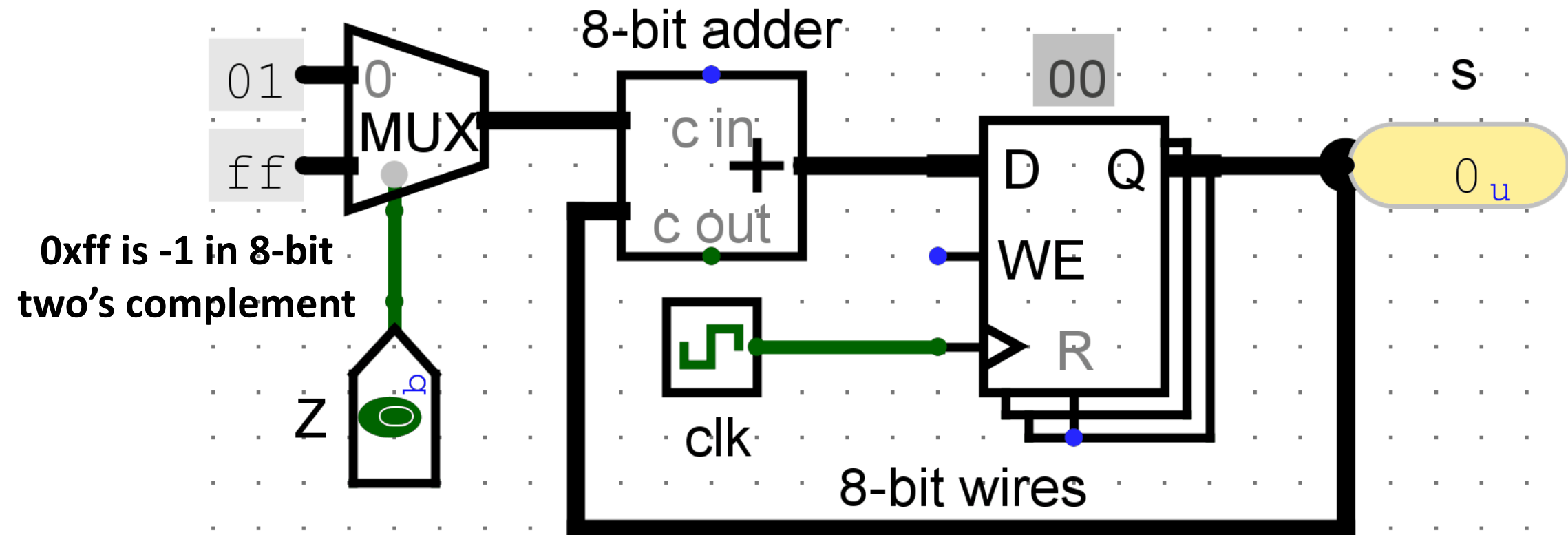


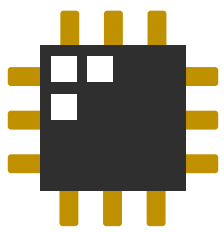
You can simply change these numbers to make it increment with any value!



Introducing more complexity

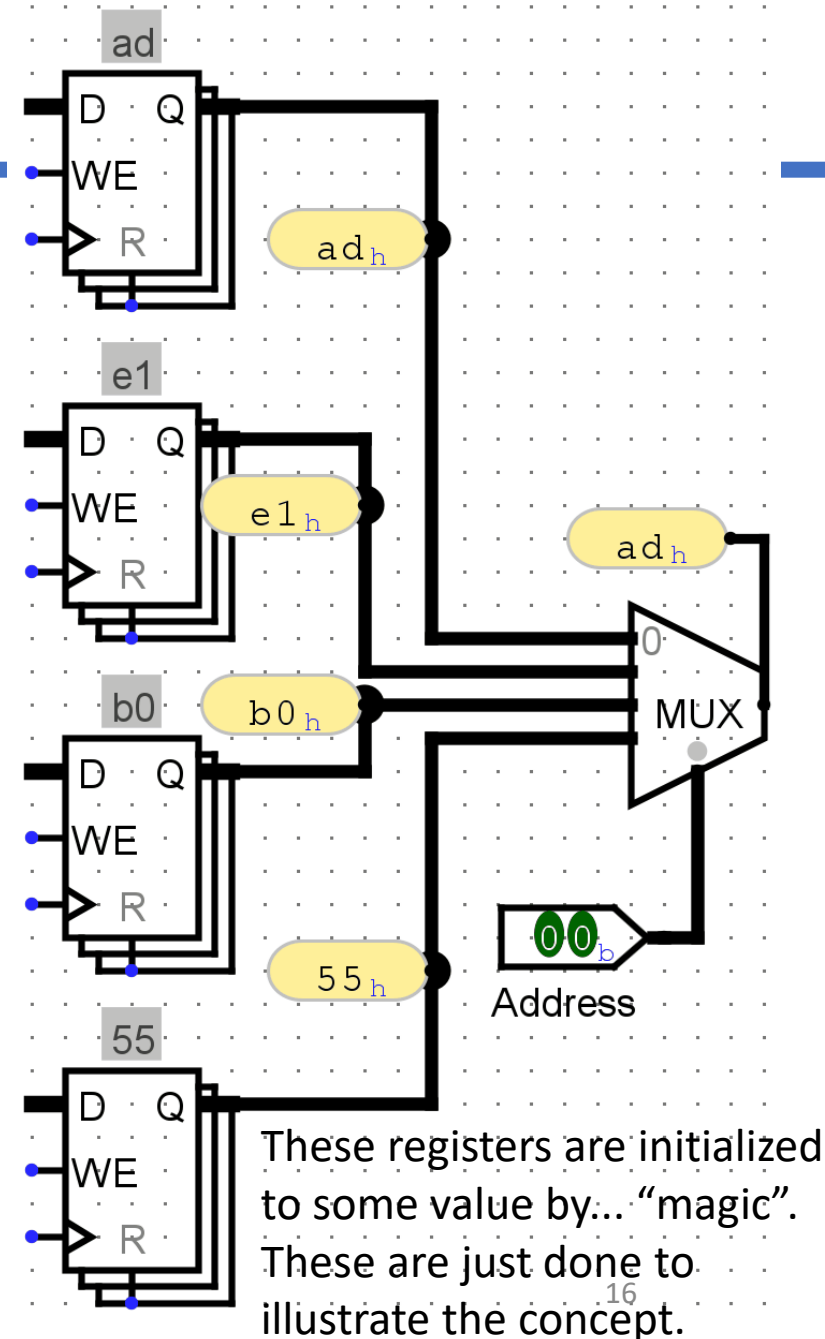
- Modify the previous accumulator by adding a new input **Z**.
 - If **Z = 0**, the accumulator increments by 1 (count up)
 - If **Z = 1**, the accumulator decrements by 1 (count down)

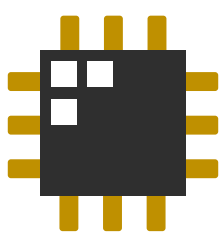




Memory structures

- Flipflops are memory elements – they hold data. You can then select from different flipflops using a multiplexer.
- You can think of memories like this
 - Flipflops are initialized to some specific value
 - All flipflop outputs go to a multiplexer for selection
 - The multiplexer selector becomes the **memory address**

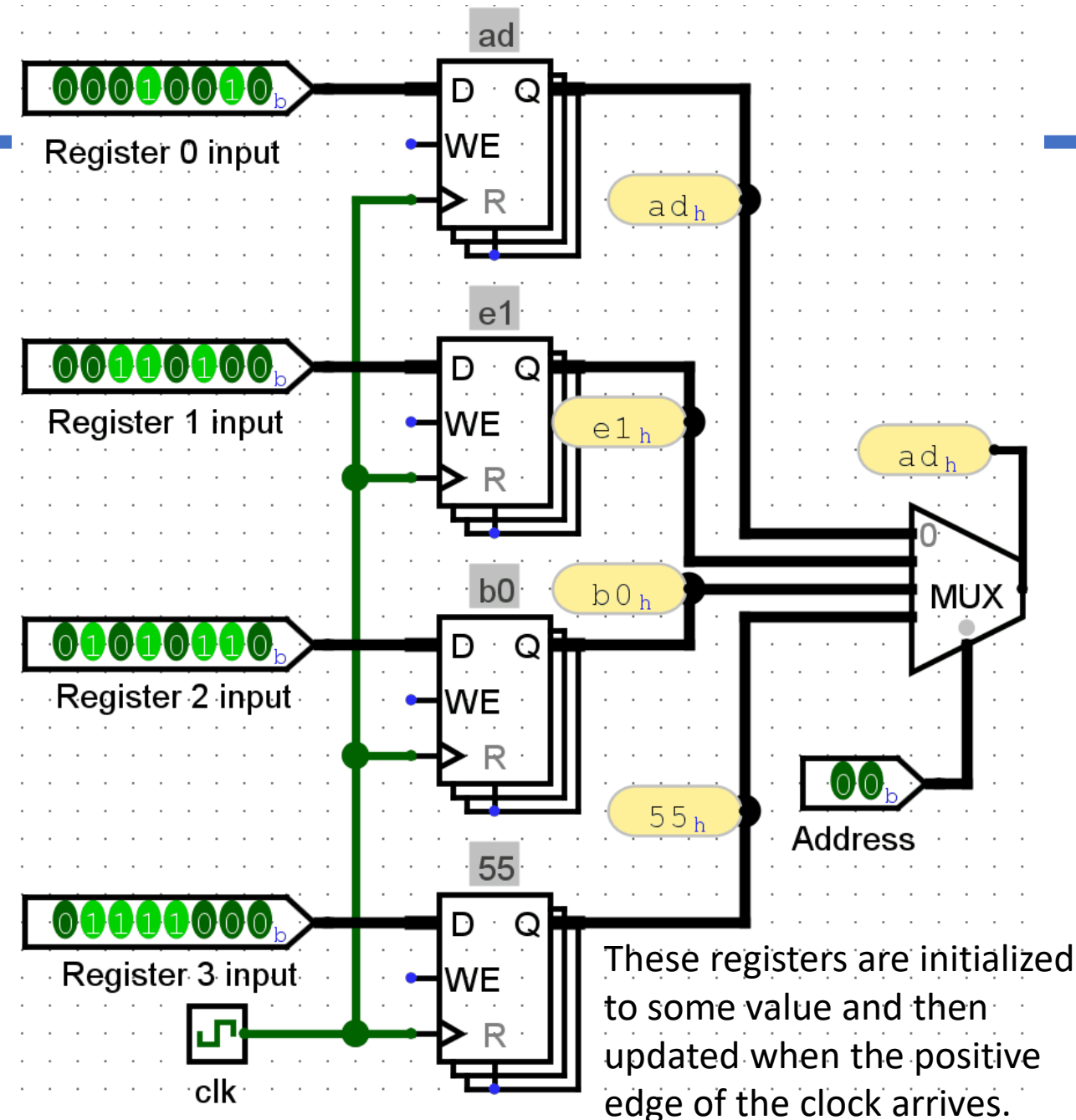


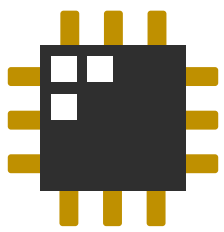


Memory structures

- If you want memories where you can **dynamically change contents**, you need to provide inputs to the flipflops. They also need a clock.
- This approach is... okay 
 - You need a lot of input though
 - Try to minimize the number of inputs by also **addressing** the writes

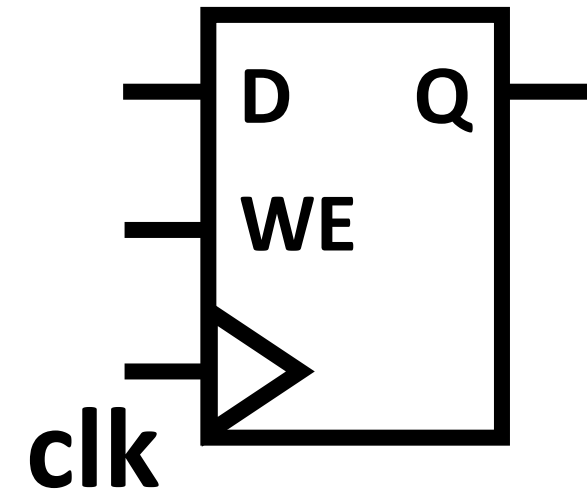
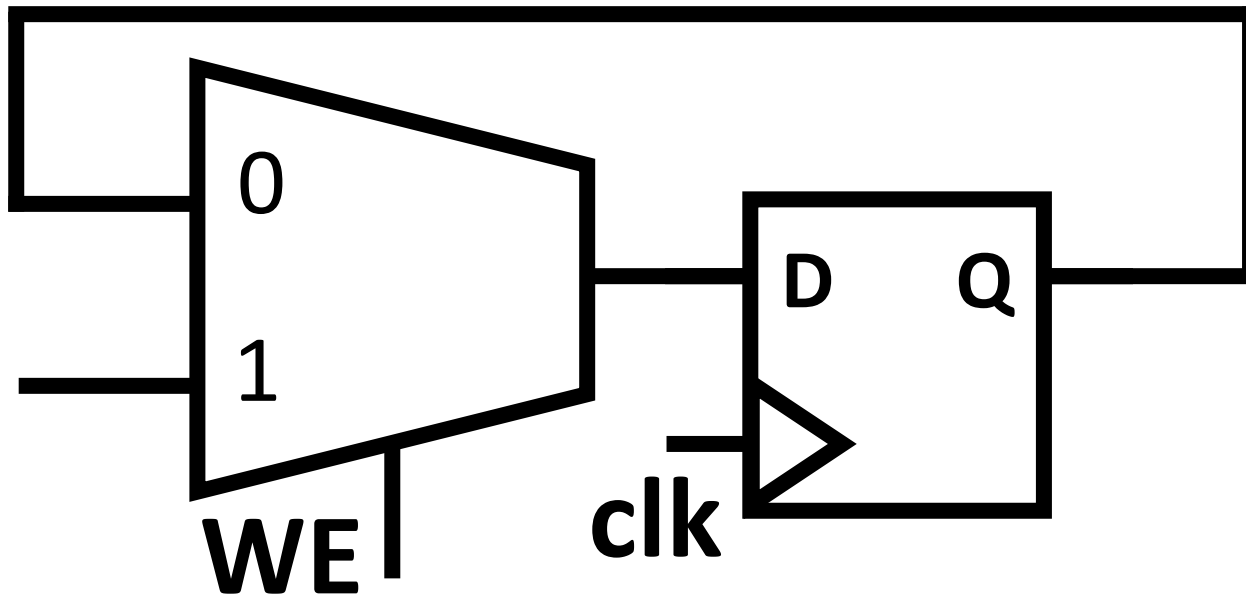
Note: Registers = flipflops with write enables
(you'll see the purpose of the write enables on the next slide!)



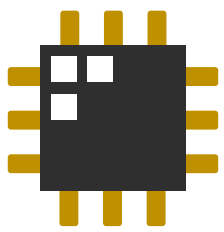


Memory structures

- Registers are flipflops with an additional input called **write enable (WE)**
 - If **WE = 1**, they behave similarly to a flipflop (update on positive edge).
 - If **WE = 0**, they just hold their values and ignore the clock completely.

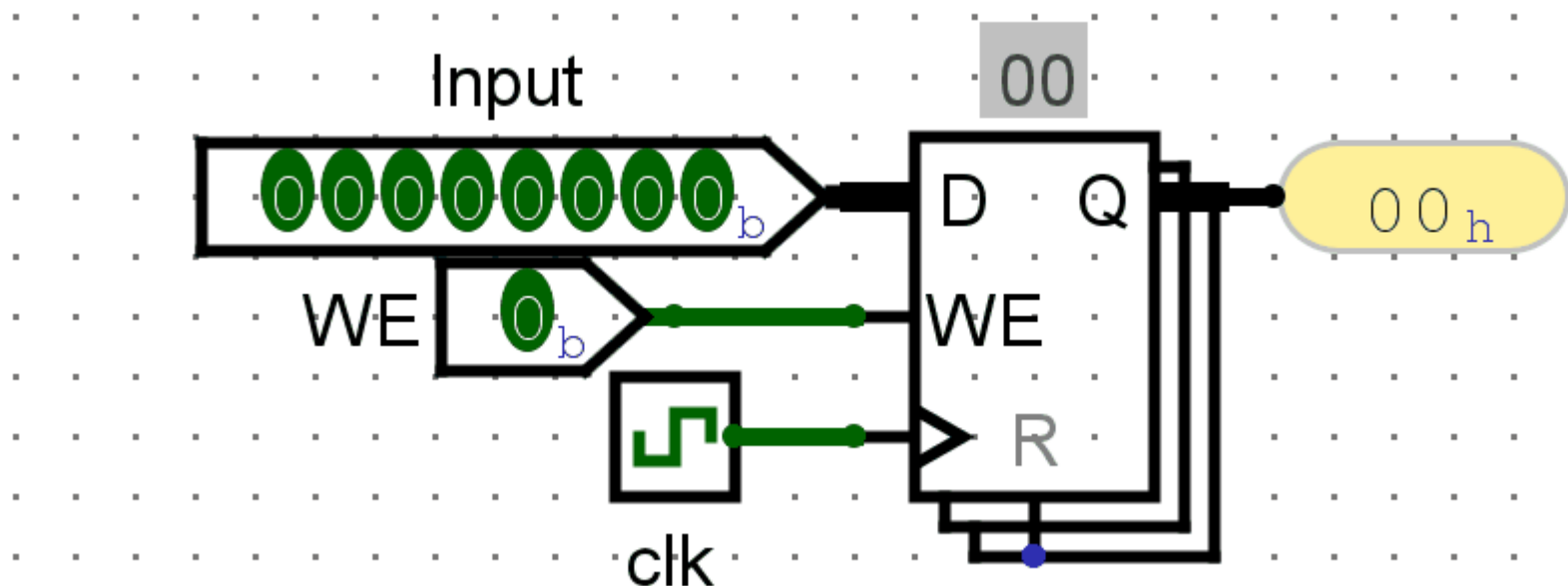


If $WE = 0$, the multiplexer provides the same input to the flipflop, making it hold the current value even with a positive edge of the clock



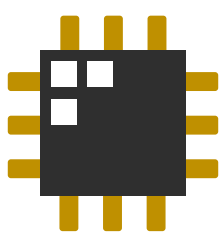
Memory structures

- Registers are flipflops with an additional input called **write enable (WE)**
 - If **WE = 1**, they behave similarly to a flipflop (update on positive edge).
 - If **WE = 0**, they just hold their values and ignore the clock completely.



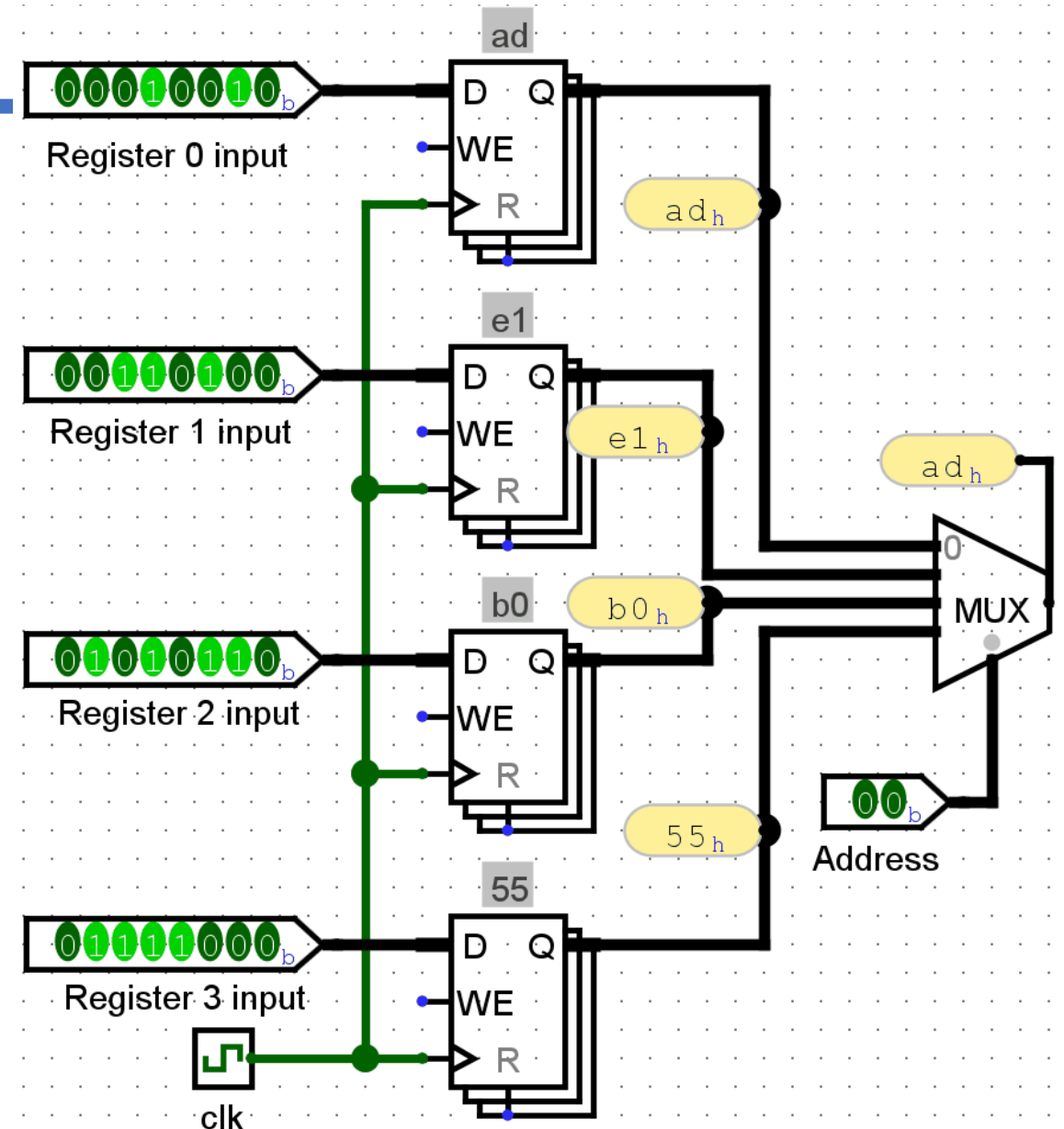
8-bit register

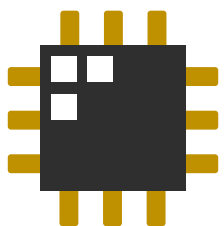
Note the behavior if WE = 0 and WE = 1.



Memory structures

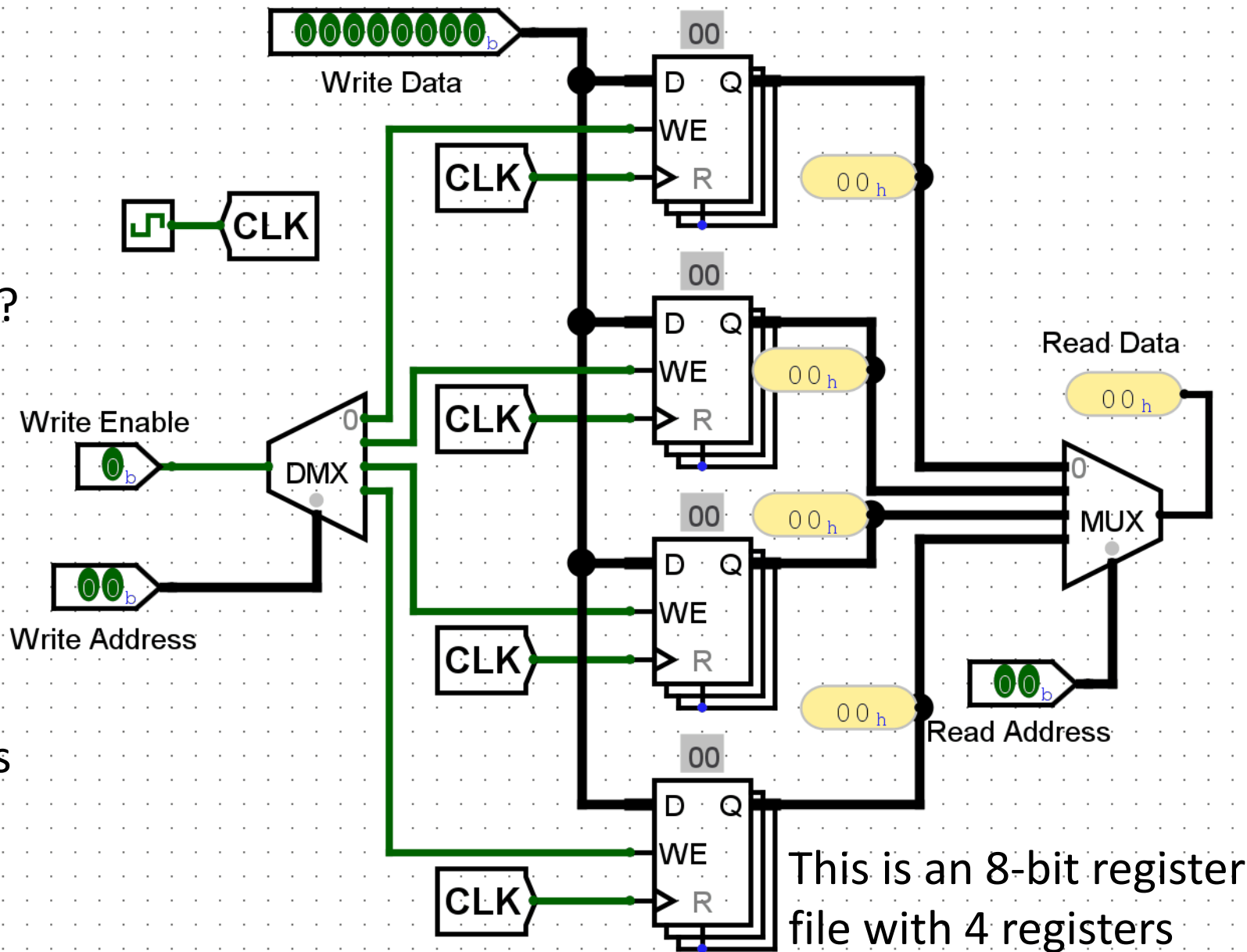
- This approach is... okay 
 - You need a lot of input though
 - Try to minimize the number of inputs by also **addressing** the writes
- Let's utilize **WE** to improve this.
 - Given an address and one 8-bit input, selectively write to only one of the four registers every positive edge of the clock.
 - We need one of the blocks introduced last meeting. *Hint: 1-to-many*

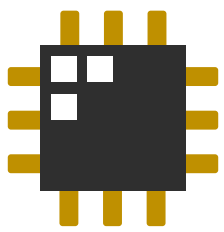




Register file

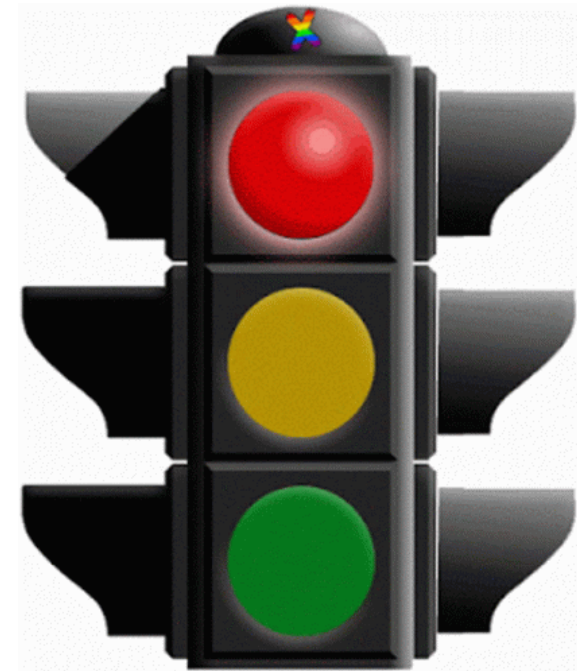
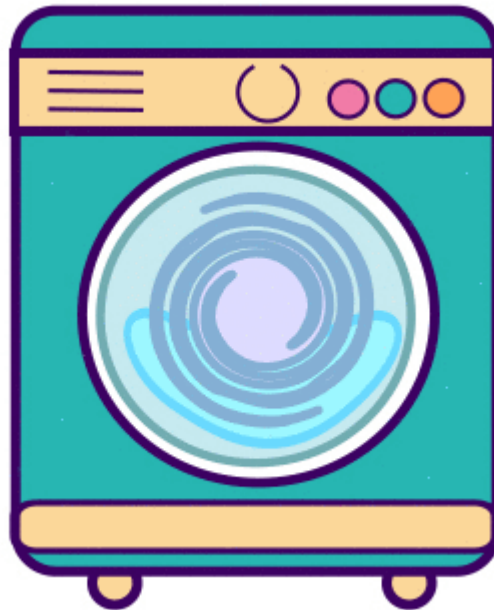
- A register file is a collection of registers
 - **Write Address:** where to write?
 - **Write Data:** what to write?
 - **Write Enable:** should I write?
 - **Read Address:** where to read?
 - **Read Data:** what was read?
 - **CLK:** The trigger for all registers
- A register file is one main component of a processor!

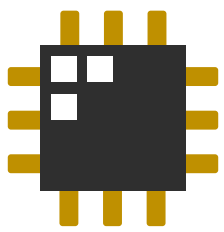




Looking forward

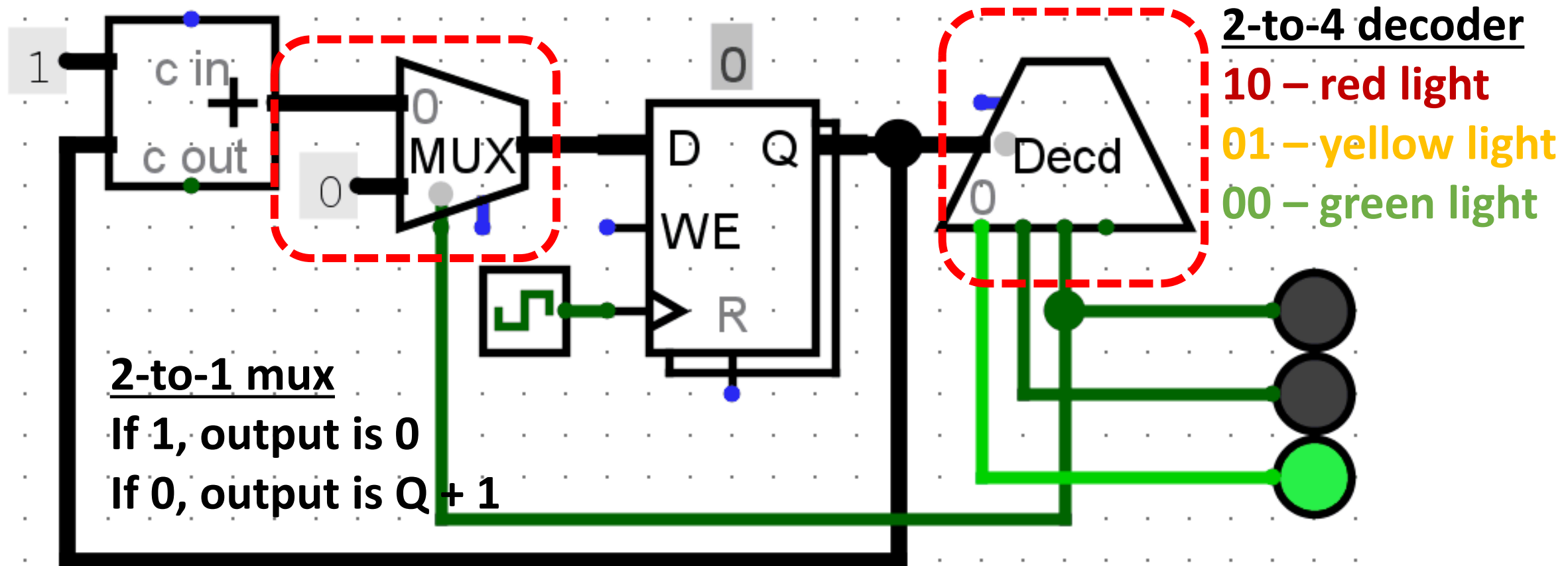
- Once we have the concept of memory to be included with logic gates, we can do more *fun* stuff
 - We can generate output *based on what happened before*.
 - We can design systems that generate a *sequence*.



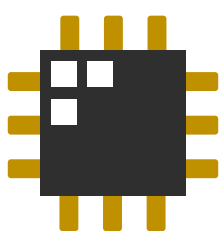


Just for fun

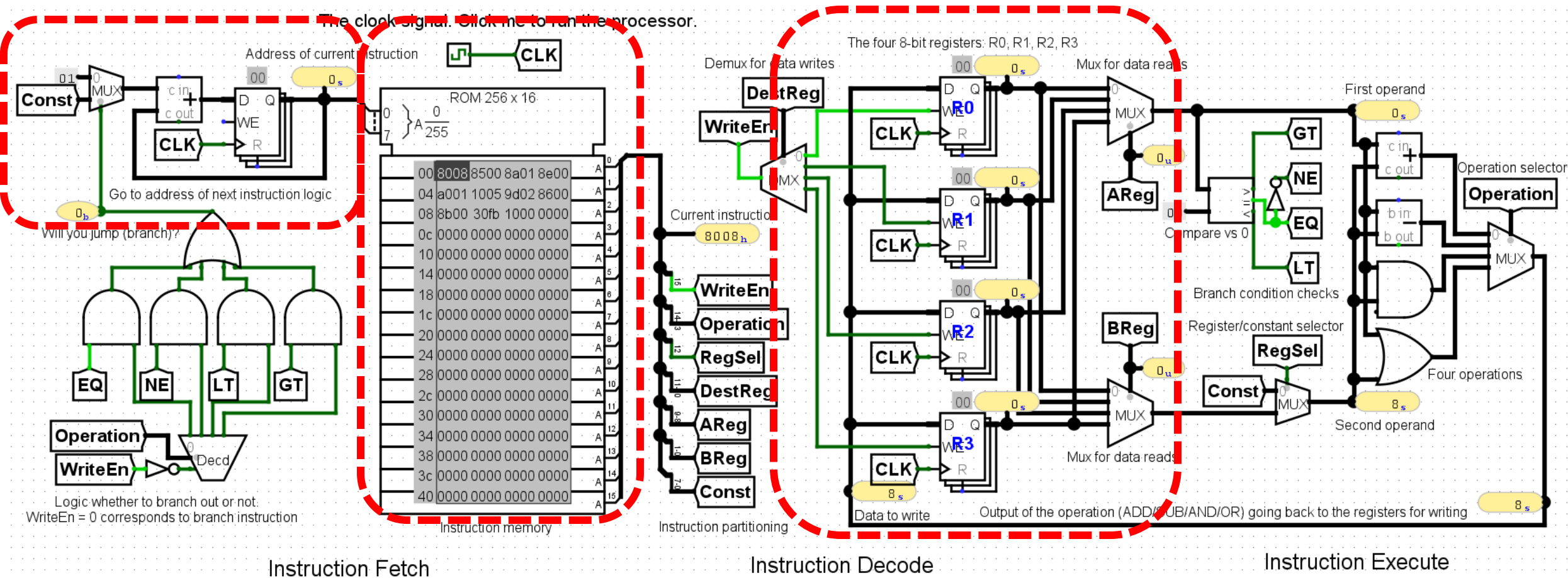
- Traffic light controller example. No inputs, just clock!







Next meeting spoilers: Your first processor



These blocks are variants of the ones we covered today!